Decision Tree Simulator

Debajit Das, Malay Mahato, Sankalan Saha

Abstract. Decision trees are among the most interpretable machine-learning models, yet the mechanics of how they choose splits are often hidden inside library calls. This report documents the Decision Tree Simulator, a web application that implements a CART-style classification tree entirely from scratch using vectorised NumPy and exposes every step of its construction through an interactive Streamlit interface. Users can select a built-in, synthetic or uploaded dataset, switch between entropy and Gini impurity, constrain the tree through four hyper-parameters, inspect the gain of every candidate split at every node, trace the path of a hand-crafted observation through the tree, and assess generalisation via confusion matrices, feature importances and accuracy-versus-depth curves. On the Iris dataset (114 training / 36 test samples), the entropy tree reaches 100% training and 88.9% test accuracy with depth 4, five leaves and nine nodes, while the Gini tree achieves 99.1% and 91.7%, illustrating the overfitting behaviour the tool is designed to expose.

Keywords: *decision tree, CART, information gain, Gini impurity, explainable machine learning, Streamlit, educational visualisation.*

1. Introduction

Tree-based models remain a cornerstone of applied machine learning because their predictions can be explained as a sequence of human-readable questions. They are also the building block of powerful ensembles such as random forests and gradient-boosted machines. Despite this, students and practitioners typically interact with trees through a single call such as fit() and see only the finished model; the reasons a particular feature and threshold were chosen at a particular node, and how those choices respond to hyper-parameters, remain opaque.

The Decision Tree Simulator addresses this gap. Its objectives are: (i) to implement the tree-induction algorithm transparently, without relying on a machine-learning library for the learner itself; (ii) to let the user change the data, impurity criterion and stopping rules and immediately observe the resulting tree; (iii) to expose the internal split search so that the gain of every candidate feature can be compared at any node; and (iv) to provide standard evaluation diagnostics that make over- and under-fitting visible. The application is delivered as a single Python file and runs in a web browser.

2. Theoretical Background

2.1 Impurity measures

A node containing n samples of k classes with class proportions p1,…,pk is characterised by an impurity value. The simulator supports two measures: Shannon entropy, $H = -\Sigma\, pi \log2\, pi$, which is zero for a pure node and log2 k for a uniform one (1.585 for three balanced classes), and Gini impurity, $G = 1 - \Sigma\, pi^2$, whose maximum is $1 - 1/k$.

2.2 Split selection

For a candidate rule “feature $j \le t$”, the samples are partitioned into a left child (nL samples) and a right child (nR samples). The gain of the split is the reduction of impurity from the parent to the size-weighted average of the children: $\text{Gain} = I(\text{parent}) - (nL/n)\cdot I(\text{left}) - (nR/n)\cdot I(\text{right})$. With entropy this quantity is the familiar information gain. Candidate thresholds for a numeric feature are the mid-points between consecutive distinct sorted values, and the split with the largest gain over all features and thresholds is selected greedily. The procedure recurses on each child until a stopping rule is met.

3. System Design and Implementation

3.1 Architecture

The application is organised in five layers within one Streamlit script: (1) a sidebar that gathers all user controls; (2) a data layer for loading and preprocessing; (3) the tree engine (impurity, split scanning, recursive growth, prediction, importance and tracing); (4) visualisation helpers that render the tree as a custom SVG and produce HTML class-distribution bars; and (5) five result tabs: Tree, Split explorer, Predict, Performance and Data. Streamlit re-runs the script on every interaction, and the expensive fit_tree function is wrapped in st.cache_data so that identical parameter combinations are served from cache.

3.2 Data layer

Four scikit-learn datasets (Iris, Wine, Breast cancer, Digits with 64 features) are bundled for convenience; scikit-learn is used only to load them. A synthetic generator produces n samples with p features, of which p_inf are informative: class-specific Gaussian mean shifts scaled by a "separation" parameter are added to the informative columns, which are then randomly permuted. The identities of the true informative features are displayed, enabling the user to check whether the tree rediscovers them. Users may also upload any CSV file and choose the target column. Preprocessing converts booleans and numerics to floats, fills missing numeric values with the column median, one-hot encodes categorical columns having at most 20 levels and drops those with more. The target is label-encoded, and a stratified train/test split with a user-chosen test fraction and seed preserves class proportions.

3.3 Vectorised tree induction

The core routine scan_feature evaluates all candidate thresholds of one feature in a single pass. The samples are sorted by the feature value; the cumulative sum of the one-hot label matrix then yields the class counts of every possible left child, and the right-child counts follow by subtracting from the node total. Impurities of all left and right partitions are computed as array operations, and positions where adjacent sorted values are equal or where a child would have fewer than min_leaf samples are masked out. The cost per feature per node is therefore dominated by the $O(n \log n)$ sort rather than by an $O(n^2)$ loop. A guard replaces a mid-point that rounds up to the upper neighbour with the lower value, so that every threshold separates the data as intended.

The recursive grow function creates a Node record (identifier, depth, sample indices, class counts, impurity, chosen feature, threshold, gain and child links), tests the stopping rules, scans all features, and keeps the best split. Ties are resolved in favour of the earlier feature because a strictly greater gain is required to replace the incumbent. Growth stops when any of the following holds: the maximum depth is reached; the node has fewer than min_split samples or fewer than 2·min_leaf samples; the node is pure; the best gain is numerically zero; or the weighted gain (gain × n_node / N) falls below min_gain, which acts as a pre-pruning threshold.

Table 1. User-controllable parameters of the simulator.

Hyper-parameter	Effect on the induced tree
Max depth (1–15)	Hard limit on the number of consecutive questions; the primary control of model complexity.
Min samples to split (2–60)	A node with fewer samples is kept as a leaf.
Min samples per leaf (1–60)	Candidate splits that create a smaller child are rejected.
Min weighted gain (0–0.10)	Split must reduce impurity, weighted by node size, by at least this amount.
Criterion	Entropy (information gain) or Gini impurity.
Test set size / seed	Fraction (0.10–0.50) and random seed of the stratified hold-out split.

3.4 Prediction, importance and path tracing

Prediction routes each sample from the root to a leaf by repeatedly testing x[feature] ≤ threshold and returns the majority class of the leaf. An optional depth limit stops routing at a given level, which allows

the accuracy of a deep tree to be evaluated as if it had been grown to a smaller depth. Feature importance is the total weighted gain contributed by each feature, Σ n_node · gain / N, normalised to sum to one. The trace method returns the list of nodes visited and the comparison made at each, which drives the Predict tab.

3.5 Visualisation

Rather than depending on Graphviz or a JavaScript library, the tree is drawn by a bespoke SVG generator. Node boxes show the split rule, impurity, sample count, class counts and majority class; the fill colour is the colour of the majority class, with saturation proportional to the purity of the node, and a horizontal stacked bar shows the class mixture. Leaves are rounded, edges are labelled “yes” (condition true) and “no”, and every node is painted with an opaque fill so that it remains legible in both light and dark themes. The layout supports top-down and left-to-right orientation, a “depth shown” control that collapses deeper sub-trees into dashed nodes, and fit-to-width or zoomed display.

4. User Interface and Features

The left sidebar (Controls) groups the settings in four steps: Data, Split criterion, Tree shape and Evaluation. When entropy or Gini is selected, the corresponding formula and the definition of gain are rendered beneath the toggle. A header banner lists the dataset, sample count, number of features, number of classes and criterion, and a row of metric cards reports train accuracy, test accuracy (with the train–test gap), tree depth, number of leaves and number of nodes. If the training accuracy exceeds test accuracy by more than ten percentage points, the application shows an overfitting warning that suggests lowering the depth or raising the minimum leaf size. All screenshots below were captured using the Iris dataset with entropy, max depth 4, min split 2, min leaf 1, min gain 0, test size 0.25 and seed 42.

4.1 Tree tab

The Tree tab renders the fitted model. Figure 1 shows the left-to-right layout for Iris: the root asks whether petal length ≤ 2.6, isolating all 38 setosa training samples in a pure leaf; the remaining 76 samples are divided by petal width ≤ 1.75, after which a further split on petal length ≤ 4.95 and a final split on sepal width ≤ 2.65 separate the few ambiguous versicolor/virginica observations.



Figure 1. Tree tab: fitted entropy tree in left-to-right layout with class counts and impurity.

4.2 Split explorer

The Split explorer lets the user select any node and examine why it was split. For the root (Figure 2), the cards show 114 samples, entropy 1.5850 (the theoretical maximum for three equally likely classes), majority class setosa and an information gain of 0.9183. The bar chart ranks the best gain achievable with each feature, highlighting the chosen one, while the table lists the best threshold per feature: petal width (threshold 0.8) and petal length (2.6) tie at 0.9183, sepal length reaches 0.5757 and sepal width

0.2732. Both petal features isolate setosa perfectly; the tie is resolved in favour of petal length, the earlier column. This view makes the greedy nature of the algorithm directly observable.

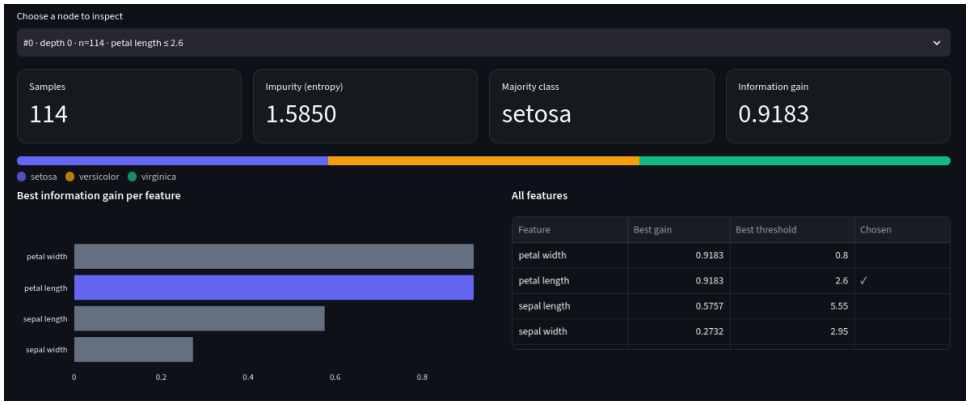


Figure 2. Split explorer for the root node: per-feature best information gain and thresholds.

4.3 Predict tab

In the Predict tab each feature has a slider (or a selector for binary features) initialised to the training median; a “Random test row” button loads a real held-out observation. The panel on the right shows the predicted class, the class distribution of the destination leaf, and a numbered decision path such as “petal length = 5.3 > 2.6 → go right”. Below, the tree is redrawn with the visited nodes and edges outlined in red (Figure 3). For the example shown, the input is classified as virginica through two comparisons.



Figure 3. Predict tab: user-supplied feature values, decision path and highlighted route.

4.4 Performance tab

The Performance tab (Figure 4) contains four diagnostics: a test-set confusion matrix with a per-class precision, recall, F1 and support table; a feature-importance chart; an accuracy-versus-depth curve for training and test data with the current depth marked; and a side-by-side comparison of entropy and Gini trees trained under identical settings. The depth curve is computed by growing one tree to depth 15 and evaluating it with truncated routing, which avoids refitting for every depth.

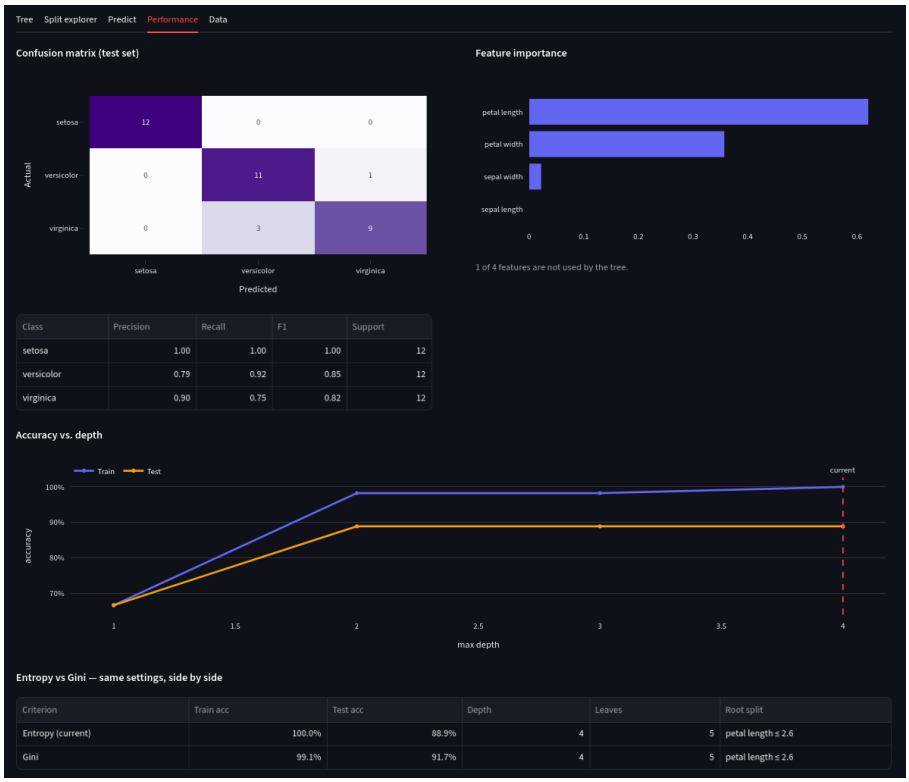


Figure 4. Performance tab: confusion matrix, per-class metrics, feature importance, accuracy vs. depth and criterion comparison.

4.5 Data tab

The Data tab (Figure 5) shows the first 200 rows of the dataset, a class-balance bar chart and descriptive statistics of the model features after preprocessing, helping the user verify data quality and class imbalance before interpreting results.

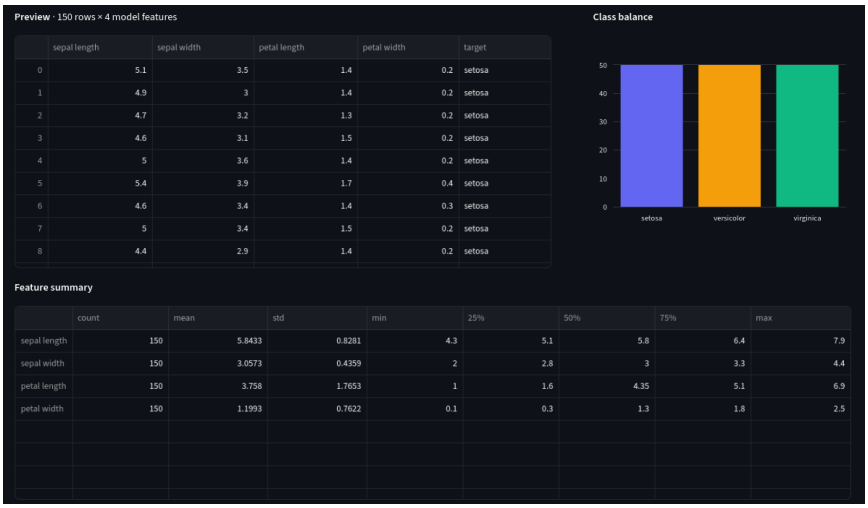


Figure 5. Data tab: dataset preview, class balance and feature summary statistics.

5. Experimental Results and Discussion

Iris contains 150 samples, 50 per class, described by four measurements. The stratified split places 114 samples in the training set (38 per class) and 36 in the test set (12 per class). Table 2 summarises the results of the two criteria under identical settings, as reported in the Performance tab.

Table 2. Entropy versus Gini on Iris (max depth 4, seed 42).

Criterion	Train acc.	Test acc.	Depth	Leaves	Root split
Entropy	100.0%	88.9%	4	5	petal length ≤ 2.6

Criterion	Train acc.	Test acc.	Depth	Leaves	Root split
Gini	99.1%	91.7%	4	5	petal length ≤ 2.6

Confusion matrix. All 12 setosa flowers are classified correctly (precision = recall = 1.00). One versicolor is predicted as virginica and three virginica as versicolor, giving versicolor precision 0.79, recall 0.92 and F1 0.85, and virginica precision 0.90, recall 0.75 and F1 0.82. The errors are confined to the two overlapping classes, which agrees with the well-known linear separability of setosa.

Feature importance. Petal length contributes about 0.6 of the total impurity reduction, petal width about 0.36, sepal width a small remainder and sepal length none; one of the four features is therefore unused. This ranking is consistent with the Split explorer, where the petal measurements dominate the gain.

Overfitting behaviour. Accuracy rises from roughly 67% at depth 1 (which can only separate setosa) to about 98% (train) and 89% (test) at depth 2, and then the test curve is flat while the training curve creeps to 100% at depth 4. The additional two levels buy no test accuracy; they merely isolate a handful of training samples (e.g. the 4-sample node split on sepal width), which is the signature of overfitting and explains the 11.1-point gap flagged by the application. Reducing max depth to 2 or raising the minimum leaf size would give a simpler tree with equal test accuracy. The difference between the criteria (88.9% versus 91.7%, i.e. one test sample out of 36) is within the noise of such a small test set, which agrees with the general observation that entropy and Gini rarely lead to materially different trees; the interactive comparison lets users verify this on other datasets and seeds.

6. Limitations and Future Work

- Only classification is supported; a regression variant would require variance-based impurity and mean-valued leaves.
- Split search is exhaustive over all features and thresholds; for the 64-feature Digits data or large synthetic sets with 20,000 samples, fitting is slower, although caching mitigates repeated runs.
- Only pre-pruning is available. Cost-complexity (post-)pruning, k-fold cross-validation and ensembles (bagging, random forests) are natural extensions.
- Categorical variables are one-hot encoded and the evaluation relies on a single hold-out split, so reported accuracies carry sampling variability.
- Missing values are imputed with the median and class weights are not supported for imbalanced data.

7. Conclusion

The Decision Tree Simulator turns the opaque process of tree induction into an explorable experience. By combining a transparent, vectorised from-scratch learner with a custom SVG renderer, split-level diagnostics, path tracing and performance analytics, it allows users to see how impurity criteria, stopping rules and data characteristics shape a model, and to recognise overfitting quickly. The Iris case study demonstrates that the tool reproduces expected behaviour: the petal measurements dominate, setosa is separated by a single split, and deeper trees improve training accuracy without improving test accuracy. The project is suitable as a teaching aid and as a basis for further work on pruning, ensembles and regression trees.